

chapter6 link layer



链路层（Link Layer）与局域网（Local Area Network, LAN）：

1. 链路层在网络层之下，解决分组**单段链路传输**问题，涉及封装为**链路层帧**（Link Layer Frame）、协议适配、广播冲突、编址配合、交换机（Switch）与路由器（Router）差异。
2. 链路层信道分两类：广播信道（Broadcast Channel）用于多主机场景（如 HFC），需介质访问协议协调；点对点信道（Point-to-Point Channel）用于设备直连，对应 PPP 协议。
3. 本章涵盖差错检测、交换局域网（如以太网 Ethernet）、虚拟局域网等，WiFi 等无线局域网留到第 7 章。

error detection, correctionsharing a broadcast channel: multiple accesslink layer
addressinglocal area networks: Ethernet, VLANs

1. 链路层概述

Nodes & Links (wired, wireless, LANs), link layer has responsibility of transferring datagram from one node to **physically adjacent node** over a link

1.1 链路层提供的服务

链路层协议能够提供的可能服务：

1. 成帧与链路接入（framing, link access）

- **成帧**：将网络层数据报封装为**链路层帧（link layer frame）**，添加首部（header）、尾部（trailer），帧结构由具体链路层协议定义。
- **链路接入**：通过**介质访问控制协议（Medium Access Control, MAC）**规定帧的传输规则：
 - 点对点链路（point-to-point link）：规则简单（或无），链路空闲即可发送。
 - 共享广播链路（shared broadcast channel）：需协调多节点的帧传输（多路访问问题）。
- **MAC 地址**：帧首部的 MAC 地址标识源 / 目的节点。

2. 可靠交付（reliable delivery）

- 实现方式：通过**确认（acknowledgments）与重传（retransmissions）**，保证数据报无差错传输。（之前：传输层TCP也可靠）

- 适用场景：多用于**无线链路（wireless links）**（高差错率，需本地纠错）。
- 局限性：低误码的 有线链路（wired links）通常不提供该服务（如以太网 Ethernet 是 “尽力而为（best-effort）” 型），避免额外开销。

3. 流量控制（flow control）

- 功能：调节相邻发送 / 接收节点的传输速率，避免接收方过载。
- 特点：链路层中**较少**使用。

4. 差错检测与纠正（error detection, correction）

- **差错检测**：
 - 成因：信号衰减（signal attenuation）、噪声等导致比特错误。
 - 机制：发送方在帧中加入检测比特，接收方检测到错误后，触发重传或丢弃帧。
 - 对比：**链路层的差错检测**比运输层 / 网络层的校验和checksum更复杂，且通过**硬件实现**。
- **差错纠正**：接收方直接识别并修正比特错误，**无需重传**。

5. 半双工与全双工（half-duplex and full-duplex）

- **半双工（half-duplex）**：链路两端节点可传输，但不能同时进行（如 LAN 集线器 hub）。
- **全双工（full-duplex）**：链路两端节点可同时收发数据（如网卡 NIC、交换机

1.2 链路层在何处实现

链路层的实现位置与载体

1. 部署范围：每个主机（host）都实现了链路层。
2. 硬件载体：链路层通过网络适配器（network adapter，又称 NIC：network interface card）。
3. 功能覆盖：NIC 同时实现了 链路层（link layer）与物理层（physical layer）的功能。
4. 主机连接：NIC 通过主机的系统总线（如 PCI：Peripheral Component Interconnect）接入主机。
5. 技术组合：链路层的实现是 硬件、软件、固件（firmware）的结合体。

接口间的通信流程

以 “发送端 - 接收端” 的交互为例：

- 发送端（sending side）：
 - a. 将网络层的数据报（datagram）封装为链路层帧（frame）；

- b. 为帧添加**差错检测比特**，并实现**可靠传输**、**流量控制**等链路层服务。
- 接收端（receiving side）：
 - a. 对收到的帧进行**差错检测**、**可靠传输校验**、**流量控制**等处理；
 - b. 从帧中提取出数据报，向上传递给接收端的上层协议（网络层及以上）。

2. 差错检测和纠正技术error detection, correction

维度	Error Detection（错误检测）	Error Correction（错误纠正）
功能	仅能发现数据是否出错	检测并自动修复错误
英文缩写	EDC（Error Detection and Correction bits）	FEC（Forward Error Correction）
应用场景	数据链路层、传输层	高可靠性需求（无线、卫星通信）
重传机制	需要重新请求发送（Retransmission）	无需重传，减少延迟
成本	低开销，少量冗余位	高开销，大量冗余位

1 奇偶校验（Parity Checking）

单比特奇偶校验（Single Bit Parity）

- 原理：添加一个校验位，使数据中1的个数为偶数（偶校验 Even Parity）
- 检测能力：✅ 只能检测单个比特错误（single bit error）

二维比特奇偶校验（Two-Dimensional Bit Parity）

- 能力升级：✅ 既能检测又能纠正单个比特错误
- 结构：同时计算行校验位（row parity）和列校验位（column parity）
- 纠正原理：错误比特位置 = 行校验失败 ∩ 列校验失败

2 校验和（Checksum / Check-Summing）

原理（典型应用于传输层）

- 计算位置：sender & receiver
- 计算方式：
 - a. 将数据分为 16比特整数段（16-bit integers）

- b. 执行1的补码求和（one's complement sum）
- c. 求和结果的反码作为校验值

关键点（选择题高频考点）

- 覆盖范围：Internet 中的 TCP 和 UDP 协议都使用 Checksum
 - TCP Checksum：覆盖 TCP header + Data + 伪头部（包含源IP、目的IP）
 - UDP Checksum：覆盖 UDP header + Data + 伪头部
 - IPv4 Checksum：仅覆盖 IP header（不包括数据）
- 可靠性：⚠ 不是100%可靠
- 选择题提示：
 - IPv4 vs TCP/UDP：IPv4只检查头部，TCP/UDP检查整个段（包括头部和数据 and 伪头部）

3 循环冗余校验（CRC - Cyclic Redundancy Check）

为什么在链路层用CRC？

- 📍 应用层次：Data Link Layer（数据链路层）适配器中最常用
- ✨ 优势：比奇偶校验更强大，比校验和更高效
- 🌐 国际标准：已定义 8位、12位、16位、32位生成元 G（Generator）

CRC的检错能力（高频考题！）

CRC类型	生成元大小	检错能力
CRC-8	9比特	能检测 ≤8比特的突发错误 (burst errors)
CRC-16	17比特	能检测 ≤16比特的突发错误

关键性质（必备！）

- ✅ 可以检测所有 r+1 以内的突发错误 (burst errors of length ≤ r+1)
- ✅ 可以检测任何奇数个比特错误
- ✅ 检测概率：对于更长的错误，检测概率为 $1 - 0.5^r$

前向纠错（FEC）的两种实现

FEC 的核心价值

- 定义：接收端同时检测与纠正错误的能力
- 好处：
 - 🚀 减少重传次数→ 降低发送端开销
 - ⚡ 即时纠正错误→ 避免往返延迟（RTT delay）
 - 📶 尤其适合：无线网络、高延迟链路
- 实现方式: FEC（前向纠错编码） Hamming Code（汉明码）

3. 多路访问链路和协议 Multiple access protocols

一、核心问题：为什么需要多路访问协议？

问题的本质

场景	问题
广播链路/共享介质	多个节点共享同一条物理链路（如老式以太网、WiFi、卫星通信）
同时传输冲突	两个或多个节点同时发送数据 → 碰撞（Collision）：接收端收到混杂的信号，无法识别
无带外协调	✗ 没有带外频道通知节点何时可以传输；所有协调必须使用信道本身进行

 Desiderata (理想特性):

1

单节点独占：当只有一个节点想传输时，它可以以满速率 R bps 独占信道

2

公平分享：当有 M 个节点都想传输时，每个节点平均分到 R/M bps

3

完全分散化：

- ✓ 没有中央节点协调（no master node）
- ✓ 没有时钟同步（no clock synchronization）
- ✓ 协议简单、廉价（simple and inexpensive）

3.1 频道分割型（Channel Partitioning）

TDMA：时间分割多路访问（Time Division Multiple Access）

原理

- 时间轴分割：信道时间分成固定长度的时隙（slots）
- 轮次：多个时隙组成一个轮次（frame/round）
- 分配：每个节点在每个轮次中获得固定的一个时隙

优点	缺点
✅ 无碰撞	❌ 节点 2、5、6 的时隙浪费 (idle slots)
✅ 公平分享	❌ 低负载时效率差：只有 $3/6 = 50\%$ 的时隙被使用
	❌ 分配不灵活：无法动态调整

FDMA：频率分割多路访问 (Frequency Division Multiple Access)

原理

- 频谱分割：信道频谱分成多个频带 (frequency bands)
- 分配：每个节点分配一个独占的频率范围
- 并行传输：所有节点可以同时传输 (在不同频率上)

优点	缺点
✅ 支持并行传输	❌ 分配的频带闲置时浪费 (如频带2、4...)
✅ 无碰撞	❌ 需要频率同步与滤波器
	❌ 低负载效率差

3.2 随机访问型 (Random Access) ★

📌 关键原则：

- 信道不被分割
- 允许碰撞 (collisions)
- 从碰撞中恢复 (使用延迟重传)
- 完全分散化：节点独立决定何时传输

Pure ALOHA (纯ALOHA) 最简单的随机访问协议

操作方式：

1. 节点有数据要发送 → 立即发送 (不等待)
2. 发送后，监听接收反馈：
 - 收到ACK → 成功✓
 - 超时 → 碰撞，延迟随机时间后重传

Slotted ALOHA (时隙ALOHA) ★改进: 加入时间同步

操作方式:

1. 所有节点时钟同步, 时间分成等长的时隙
2. 节点只能在时隙开始时发送数据
3. 如果碰撞, 在后续时隙以概率 p 重新传输



假设 $N=3$ 个节点, 都有数据, 各以概率 p 发送

时隙 1: [node1发] [node2发] [node3空] → 碰撞✗ (nodes 1,2)

时隙 2: [node1空] [node2重] [node3发] → 成功✓ (node 3)

时隙 3: [node1重] [node2空] [node3空] → 成功✓ (node 1)

时隙 4: [node1空] [node2重] [node3空] → 成功✓ (node 2)

CSMA: 载波侦听多路访问 (Carrier Sense Multiple Access) ★★

核心创新: "听前说" (Listen Before Talk)



CSMA 操作

- 1 如果信道空闲 → 发送整个数据帧
- 2 如果信道忙 → 等待直到信道空闲, 然后发送

问题: 为什么 CSMA 还会发生碰撞? ⚠

关键: 【传播延迟 (Propagation Delay)】

CSMA/CD: 带碰撞检测 (Collision Detection) ★★★

核心思想: 尽早发现碰撞, 立即停止发送

正在发送时同时监听是否有其他节点也在发送。一旦检测到碰撞, 立即停止发送并发送干扰信号。



二进制指数退避详解 (实际时间量是 $K \cdot 512$ 比特时间)

第1次碰撞: $K \in \{0,1\}$ → 等待 0 或 512 比特时间

第2次碰撞: $K \in \{0,1,2,3\}$ → 等待 0, 512, 1024, 或 1536 比特时间

第3次碰撞: $K \in \{0,1,2,3,4,5,6,7\}$ → 等待 0 到 3584 比特时间

...

第m次碰撞： $K \in \{0,1,...,2^m-1\}$

越往后，等待时间越长，降低再次碰撞的概率

如果 $m > 16$: 放弃发送，通知上层错误

碰撞检测的难点

介质	难度	原因
有线（有线以太网）	✔ 容易	信号衰减明显，易于检测；双工传输简单
无线（WiFi）	✘ 困难	发射信号强度远大于接收信号，掩盖碰撞；隐藏节点问题

3.3 轮询型（Taking Turns）

 需求背景

【低负载】	【高负载】
└─ 随机访问 ✔	└─ 随机访问 ✘
效率好	碰撞多
延迟小	效率低
└─ 轮询 ✘	└─ 轮询 ✔
过度开销	公平、高效

理想方案：既能低负载高效，又能高负载有序！

方案1：Polling（轮询）

优点	缺点
✔ 消除碰撞	✘ 轮询开销（polling overhead）：邀请消息占用信道
✔ 高负载效率高	✘ 延迟：如果 M 个节点，某个节点要等待 M-1 个节点的轮询周期
✔ 公平分配	✘ 单点故障：Master 宕机整个系统瘫痪

方案2: Token Passing（令牌传递）☆☆

原理

- 物理令牌（Token）在节点间依次传递
- 持有令牌的节点有发送权
- 发送完毕后传递给下一个节点

优点	缺点
✓ 消除碰撞	✗ 令牌开销：额外的令牌帧占用信道
✓ 公平分配	✗ 延迟：等待令牌到达
✓ 完全分散化	✗ 单点故障：令牌丢失整个系统停止
	→ 需要令牌恢复协议

实际应用

- 令牌环网络（Token Ring）：IEEE 802.5
- Bluetooth：点对点令牌
- FDDI：光纤分布式数据接口

3.4 核心知识速记卡

📦 🎯 关键点不能忘：

【效率排行】

纯ALOHA(18%) < 时隙ALOHA(36.8%) < CSMA(~70%) < CSMA/CD(~95%)

【协议选择】

- 有线以太网 → CSMA/CD ☆
- WiFi → CSMA/CA（避免碰撞） ☆
- 蜂窝网络 → TDMA/FDMA
- 令牌网络 → Token Ring

【CSMA/CD 必背】

1. 侦听 → 2. 发送 → 3. 检测碰撞 → 4. 指数退避重试

【效率公式】

$\eta = 1 / (1 + 5a)$ ，其中 $a = d_{\text{prop}} / t_{\text{trans}}$

【二进制退避】

第 m 次碰撞： $K \in \{0, 1, \dots, 2^m - 1\}$ ，等待 $K \times 512$ 比特时间

【单点故障】

轮询：Master 宕机

令牌：令牌丢失（需要令牌恢复协议）

4. 交换局域网（LANs: Local Area Networks）

从MAC地址到VLAN的完整体系

4.1 核心问题：为什么需要 MAC 地址？

维度	IP 地址	MAC 地址
层次	第3层（网络层）	第2层（链路层）
功能	端到端路由转发	本地相邻节点通信
作用范围	全网（全Internet）	同一子网/LAN
格式	32位（IPv4），通常点分十进制	48位，16进制
例子	128.119.40.136	1A-2F-BB-76-09-AD
分配方式	分层、基于子网、可变	固定烧入ROM（大多数）
可移植性	✗ 更换子网后变化	✓ 物理地址可移植

4.2 MAC 地址详解

十六进制表示法（Base-16）：每个"数字"代表 4 比特 1A-2F-BB-76-09-AD

关键性质：MAC 地址的 "平坦性"（Flat Address Space）

  优点：

- 可移植性：可以将网卡从一个LAN移到另一个LAN
- 即插即用（Plug-and-Play）：无需重新配置

 缺点：

- 不能用于 Hierarchical Routing
- 如果全Internet都用MAC地址，没有地址聚集
- 可扩展性差（无法像IP地址那样按地理位置分级）

【结论】：这就是为什么需要IP地址（分层）+ MAC地址（本地）的组合

4.3 ARP 协议（Address Resolution Protocol）

问题：如何从 IP 地址获得 MAC 地址？（多个网络，到同一链路）

ARP 工作流程（4步）

Step 1: 创建 ARP Query（查询）广播

 A 广播一个 ARP Query 请求：

【ARP 请求帧】

目的MAC地址： FF-FF-FF-FF-FF-FF （广播地址 ← 所有设备都收到！）

源MAC地址： 74-29-9C-E8-FF-55 （A的MAC）

协议类型： ARP

【ARP 请求消息体】

源IP地址： 137.196.7.23 （A的IP）

源MAC地址： 74-29-9C-E8-FF-55 （A的MAC）

目标IP地址： 137.196.7.14 （B的IP ← 我要找这个IP！）

目标MAC地址： ???????? （待填，现在未知）

【广播方式】所有LAN上的节点都收到这个请求

Step 2: 所有节点接收并处理

Step 3: 目标节点 B 回复 ARP Response 单播

 B 发送 ARP 回复（单播，仅回给 A）：

【ARP 响应帧】

目的MAC地址： 74-29-9C-E8-FF-55 （A的MAC地址 ← 单播！）

源MAC地址： 58-23-D7-FA-20-B0 （B的MAC）

协议类型： ARP

【ARP 响应消息体】

源IP地址： 137.196.7.14 （B的IP）

源MAC地址： 58-23-D7-FA-20-B0 （B的MAC ← 答案！）

目标IP地址： 137.196.7.23 （A的IP）

目标MAC地址： 74-29-9C-E8-FF-55 （A的MAC）

Step 4: A 收到回复并更新 ARP 表

 A 收到 B 的回复后：

【ARP 表（缓存）】

IP地址	MAC地址	TTL
137.196.7.14	58-23-D7-FA-20-B0	500 秒 ← 新增
137.196.7.23	74-29-9C-E8-FF-55	500 秒
137.196.7.88	0C-C4-11-6F-E3-98	500 秒

 A 现在可以创建以太网帧发给 B

ARP 表的特点

特性	说明
内容	<IP地址, MAC地址, TTL> 三元组
作用	缓存IP-MAC映射, 避免重复ARP查询
TTL	通常 20 分钟; 超时后条目删除
范围	同一LAN内 的IP-MAC映射
更新	每次收到ARP消息 (请求或回复) 都会学习

4.4 跨子网路由: IP + MAC 地址的协同 (重要案例!)

场景: A 在子网 1 发送数据给 B 在子网 2 (需要经过路由器 R)

🍷 拓扑图:

IP地址:

A: 111.111.111.111 B: 222.222.222.222

R-左: 111.111.111.110 R-右: 222.222.222.220

Step 1: A 创建数据包 (IP层)

🍰 【IP 数据包头部】

源IP: 111.111.111.111 ← 不变!

目的IP: 222.222.222.222 ← 最终目标, 不变!

Step 2: A 创建以太网帧 (MAC层)

🌟 【问题】A 不知道 B 的 MAC 地址

A 知道路由器 R 的 IP (111.111.111.110)

❓ 该使用谁的 MAC 地址?

【答案】使用【路由器 R 的 MAC 地址】作为帧的目的地!

【关键点】：

IP地址：源 A → 目的 B（端到端不变）

MAC地址：源 A → 目的 R（逐跳变化！）

【以太网帧】

MAC 源： 74-29-9C-E8-FF-55 ← A 的 MAC

MAC 目的： E6-E9-00-17-BB-4B ← R 的 MAC（通过 ARP 获得）

IP 源： 111.111.111.111

IP 目的： 222.222.222.222

Step 3: 路由器 R 处理帧



【路由器收到帧】

检查 MAC 目的地 → 是我 ✓

【提取 IP 数据包】

检查 IP 目的地 → 222.222.222.222

【查询路由表】

222.222.222.0/24 → 从右接口转发

【创建新的以太网帧】

MAC 源： 1A-23-F9-CD-06-9B ← R 右接口的 MAC

MAC 目的： 49-BD-D2-C7-56-2A ← B 的 MAC（通过 ARP 获得）

IP 源： 111.111.111.111 ← 不变！

IP 目的： 222.222.222.222 ← 不变！

【关键观察】

✓ IP 地址头保持不变（端到端）

✗ MAC 地址头被替换（每跳一次）

Step 4: B 接收数据包



【B 接收以太网帧】

检查 MAC 目的地 → 是我 ✓

【提取 IP 数据包】

源 IP： 111.111.111.111 ← 原始发送者 A

目的 IP: 222.222.222.222 ← 我自己

✅ 正确识别来自 A 的数据

易错点警告 ⚠️

- 📌 ❌ 常见错误1: 认为 A 直接用 B 的 MAC 地址发送
 - ✅ 实际: A 用 R 的 MAC 地址, 因为 B 不在同一 LAN
- ❌ 常见错误2: 认为 IP 地址会改变
 - ✅ 实际: IP 地址头从 A 到 B 保持不变 (端到端), 只有 MAC 地址头在每跳改变
- ❌ 常见错误3: 认为路由器需要 ARP 查询 B 的地址
 - ✅ 实际: 路由器在右子网内 ARP 查询 B 因为 B 在右子网, 不是直接邻接
- ❌ 常见错误4: 搞混了"转发表"和"ARP表"
 - 转发表: IP→接口 (路由决策)
 - ARP表: IP→MAC (本地映射)

4.5 以太网 (未完待续补充)

现状:

- ✅ 最主流的有线 LAN 技术
- ✅ 从 10 Mbps → 400 Gbps (速度增长 40000 倍!)
- ✅ 单一芯片支持多种速率 (如 Broadcom BCM5761)

4.6 以太网交换机 (Ethernet Switch) ★★★★★



【什么是交换机?】

链路层设备 (Layer 2 Device)

- 存储、转发以太网帧
- 检查接收帧的 MAC 目的地址
- 有选择地转发帧到正确的出口链路

【对比: 集线器 (Hub) vs 交换机】

集线器 (Hub) :

- └— 物理层设备
- └— 接收帧 → 在所有接口转发 (广播)
- └— 所有连接的设备共享一个碰撞域
- └— 已淘汰

交换机 (Switch) :

- └— 链路层设备
- └— 检查 MAC 地址 → 选择性转发
- └— 每个接口是独立的碰撞域
- └— 现代标准 ✓

交换机工作原理：3 步流程

Step 1: 学习 (Learning)



当交换机接收到一个帧时：

【记录信息】

记录：发送节点的 MAC 地址 + 接收帧的接口

【添加到转发表】

交换机表 (Switching Table) :

MAC地址	接口	TTL (超时时间)
-------	----	------------

1A-2F-BB-76-09-AD	接口1	5分钟
-------------------	-----	-----

58-23-D7-FA-20-B0	接口3	5分钟
-------------------	-----	-----

【这就像"自学习"】

✓ 每收一个帧，就学习一个新的 MAC-接口映射

✓ 不需要管理员手动配置！ (Plug-and-Play)

✓ TTL 过期后条目自动删除

Step 2: 转发/过滤 (Forwarding/Filtering)



收到帧后，查表的三种情况：

【情况1：目的MAC在表中找到】

```
if (目的地址在表中) {  
    if (来源接口 == 目的接口) {  
        【丢弃帧】 ← 源和目的在同一网段，无需转发  
        (Frame would only propagate one hop anyway)  
    } else {  
        【从指定接口转发帧】  
        (Forward to the interface in the table)  
    }  
}
```

【情况2：目的MAC在表中找不到】

```
else {  
    【洪泛 (Flooding) 】  
    从除了来源接口外的所有接口转发  
    (Forward on ALL interfaces except arriving interface)  
    为什么？ → 广播式查找，希望目的地设备收到  
}
```

【情况3：目的MAC是广播地址 FF-FF-FF-FF-FF-FF】

【洪泛】 从除了来源接口外的所有接口转发

Step 3: 自学习示例

交换机 vs 路由器：关键差异（高频考题！）

维度	交换机 (Switch)	路由器 (Router)
工作层	第2层 (链路层)	第3层 (网络层)
检查地址	MAC 地址	IP 地址
转发表生成	自学习 (flooding + learning)	路由算法 (RIP, OSPF, BGP)
地址类型	平坦 MAC 地址	分层 IP 地址
跨越	不能跨越路由器 (同一-LAN)	可跨越多个网络
配置	✅ 即插即用	❌ 需要配置
透明性	透明 (主机无需知道)	不透明 (主机需知道)
速率	✅ 快 (仅检查MAC)	❌ 较慢 (需查询路由表、计算)
处理时间	✅ 较低 (硬件处理)	❌ 较高 (可能涉及CPU)
广播风暴	❌ 易遭受 (洪泛所有端口)	✅ 隔离 (不转发广播)

易错点

- ❌ 常见错误1: 认为交换机有"路由表"
 - ✅ 实际: 交换机有"转发表", 通过自学习生成
- ❌ 常见错误2: 认为交换机可以连接不同子网
 - ✅ 实际: 交换机只能连接同一子网的设备
需要路由器才能连接不同子网
- ❌ 常见错误3: 认为交换机需要配置 IP 地址
 - ✅ 实际: 交换机是 Layer 2 设备, 不需要 IP
管理交换机可能需要 IP, 但转发不需要
- ❌ 常见错误4: 认为交换机可以防止广播风暴
 - ✅ 实际: 交换机会洪泛未知 MAC 的帧到所有端口
防止广播风暴需要 VLAN 或生成树协议
- ❌ 常见错误5: 认为转发表中的条目永久保存
 - ✅ 实际: TTL 过期后自动删除
目的: 适应网络拓扑变化

4.7 多交换机互联 (Interconnecting Switches)

 ❌ 常见错误1：认为多交换机级联需要特殊配置

✅ 实际：完全自学习，无需配置！

❌ 常见错误2：认为交换机间的链接需要特殊处理

✅ 实际：交换机间的链接也只是普通的以太网链接

❌ 常见错误3：认为洪泛会一直进行

✅ 实际：一旦某个交换机学到了目的地 → 选择性转发
洪泛的性能消耗会迅速降低

4.8 虚拟局域网（VLAN）★★

问题：为什么需要 VLAN？

【场景】：大学校园网

CS系 40 台电脑 + EE 系 40 台电脑

都连接到同一个物理交换机

【问题1：广播域太大】

【问题2：安全隔离困难】

【问题3：管理不灵活】

? 解决方案 → VLAN!

VLAN 的基本概念

【VLAN = Virtual LAN】

单个物理交换机 → 看起来像多个虚拟交换机

【例子】

物理交换机（16个接口）

|

├── VLAN 100 (CS): 接口 1-8 ← 虚拟交换机1

└── VLAN 200 (EE): 接口 9-16 ← 虚拟交换机2

【关键特性】

- ✓ 流量隔离：VLAN 100 的帧不会转发到 VLAN 200
- ✓ 广播域分隔：每个 VLAN 有独立的广播域
- ✓ 安全性：不同 VLAN 间无法直接通信（需要路由器）

VLAN 的分类

A. 基于端口的 VLAN（Port-Based VLAN）★

B. 其他 VLAN 类型（了解）



【基于 MAC 地址的 VLAN】

- └ 根据源 MAC 地址分配 VLAN
- └ 优点：用户物理移动后自动分配
- └ 缺点：需要维护 MAC→VLAN 映射表，开销大

【基于 IP 地址的 VLAN（Layer 3）】

- └ 根据源 IP 地址分配 VLAN
- └ 优点：更灵活
- └ 缺点：交换机需要检查 IP 地址（层3），性能影响

【基于协议的 VLAN】

- └ 根据网络层协议（IPv4, IPv6, IPX 等）分配
- └ 较少使用



易错点警告 ⚠

✗ 常见错误1：认为不同 VLAN 之间可以直接通信

✓ 实际：不同 VLAN 间隔离，需要路由器才能通信

Trunk 只转发同一 VLAN 的帧

✗ 常见错误2：认为 802.1Q 标签是永久的

✓ 实际：Trunk 端口添加标签，Access 端口移除标签

用户接收到的帧不包含标签

✗ 常见错误3：认为 VLAN ID 可以任意大

✓ 实际：VLAN ID 是 12 比特，范围 0-4094（共 4095 个）

0 和 4095 有特殊含义，实际可用：1-4094

✗ 常见错误4：认为 VLAN 完全消除了广播流量

✓ 实际：只是将广播域分割

同一 VLAN 内的广播仍然存在

不同 VLAN 间仍需要路由器

✗ 常见错误5：认为 VLAN 可以跨越多跳自动工作

✓ 实际：需要中间所有交换机都支持并配置 VLAN

如果一个交换机不支持，VLAN 会被打破

4.10 选择题高频考点



选择题

? 题型3：交换机 vs 路由器

Q: 下列哪个陈述正确？

- ✗ 交换机可以连接多个不同的子网
- ✓ 路由器可以连接多个不同的子网
- ✗ 交换机可以防止所有广播
- ✓ 路由器可以防止广播转发到其他网络
- ✓ 交换机可以 Plug-and-Play，无需配置
- ✗ 路由器可以 Plug-and-Play（需要配置IP等）

? 题型6：交换机洪泛

Q: 交换机何时会洪泛（Flood）一个帧？

- ✓ 当帧的目的 MAC 不在转发表中
- ✓ 当帧的目的 MAC 是广播地址（FF-FF-FF-FF-FF-FF）
- ✓ 当帧的目的 MAC 是多播地址
- ✗ 当帧的源 MAC 不在表中（不洪泛，但会学习）

? 题型7：802.1Q 标签

Q: 802.1Q VLAN 标签包含什么信息？

- ✓ VLAN ID（12 比特）
- ✓ 优先级/优先权（3 比特，类似IP的TOS）
- ✗ MAC 地址（不包含）

5. a day in the life of a web request

6. MPLS (Multiprotocol Label Switching)

7. VPN